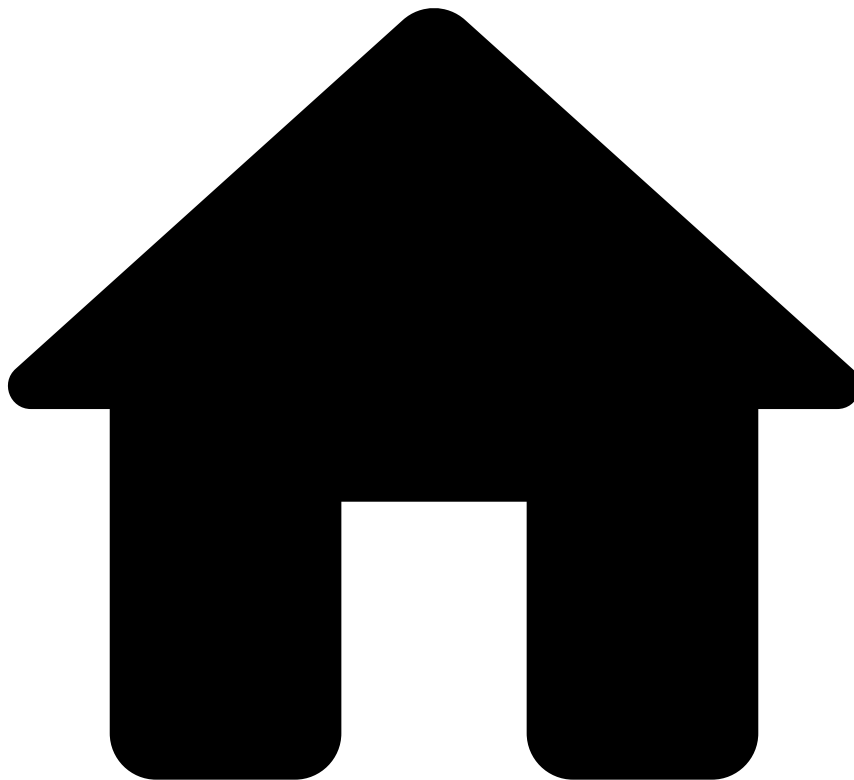


•



- Best Practices
- [Metrics](#)
- Constraints

On this page

Was this page helpful? 

Constraints

Although Pulse uses the same feature definition offline (Data Science Platform) and online (Runtime Environment), there are important distinctions between them.

Development vs. live scoring

This distinction means that as a best practice, runtime environment constraints should be known upfront before creating metrics manually.

In the following sections you can find the most common constraints when developing metrics.

Runtime Environment📄

When processing high throughputs in runtime (e.g. 1000 TPS at peak), it is advisable to have **runtime partitioning**. This means that events will be processed by different machines, according to a partitioning strategy. The most common strategies are to partition by card/account. This way it is guaranteed that all events from cards are processed in **real time** by the same machine.

So when designing **real time metrics** you must be aware if partitioning is active in your runtime environment. If it is, and the metric does not contain said key as the root grouping entity, there will be a discrepancy in the calculations between the Data Science and Runtime environments. Note that **this does not affect scheduled metric jobs (SMJs)**. Because SMJs are applied to events fetched from the Event Storage, they will always see all the events that are processed, independently of the partitioning key. For more details on SMJs best practices, go to the [Scheduled Metrics Jobs \(SMJs\)](#) page.

The following table summarizes the points to keep in mind when working with metrics:

	Context	Implication
Runtime Partitioning	When data is partitioned in runtime, each node will only see a subset (partition) in real time. Partitioning is needed to process large volumes (rule of thumb: >500 events per second).	Real-time metrics must use: 1. The same field as the partitioning key as the root grouping entity. 2. Or, an entity with its domain within the partitioning key.
Scheduled Metrics Jobs (SMJs)	Metrics with long windows run in batch jobs in runtime. These batches are scheduled to run at specific times to not overload the system.	Re-running the plan with the same data in the Data Science environment will lead to small errors (up to 1% mismatch in value) in a few samples of data (approx. 1% of samples, depending on job configuration and cluster capacity).
Optimizations in SMJs	Because SMJs are special jobs that fetch data from storage and write the results as a key-value pair, there are a few optimizations done that are not possible in the Data Science Platform.	Null values in grouping entities are excluded from the SMJ. This helps reduce failure risk due to skewness. Distinct aggregations are one-shot, meaning that the memory consumption is much less than in runtime.

Data Science Platform📄

Within the data science platform (offline) there are different considerations to take into account. The next table highlights scalability concerns that are relevant when executing plans. In these plans an input data source is processed, metrics are computed on top of it, and the resulting data source is persisted.

	Context	Implication
Spark Jobs & data volume	Metrics done in the Data Science Platform make use of Spark to distribute the workload across an Hadoop cluster. Distributing metrics through a Spark job implies data will be moved around in the cluster in large quantities. Due to the nature of profiles, each value of an entity (e.g. each card or account) will be processed in the same Spark machine (or executor).	Even though a metric might only need 2 or 3 fields to compute, Spark will still move the records with all their columns to the executors.
Spark Jobs & skewness	Very high skews in grouping entities (e.g. 10% of entities represent 90% of the total records), can make Spark Tuning more complex.	Avoid including highly skewed grouping entities in your metrics.

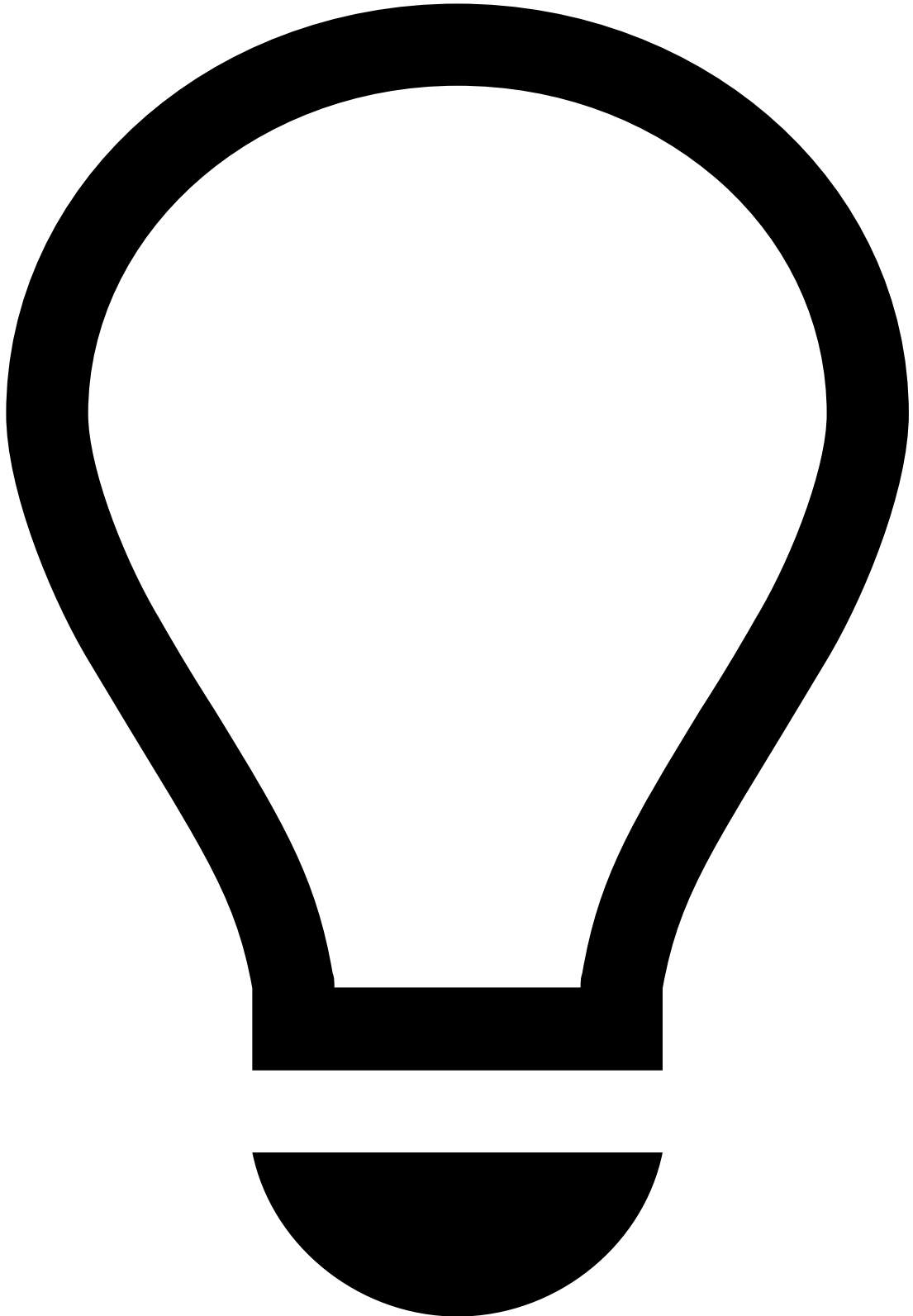
	Context	Implication
		If they cannot be avoided, there are two main avenues to solve it: • Randomize special entities (eg. govt, gmail, ...) • Salt the largest entities: added cost of then joining the features
Window sizes	In the Data Science Platform, all windows are processed through Spark. Even though the SMJs effect is simulated, a large window will still be computed "in-memory" inside the Spark executors.	Very large windows might be feasible in runtime, but not in Data Science Platform.

Learn more about the implications of distributed Spark jobs within the Data Science Platform on the [data partitioning](#) page.

Learn more about the implementation of [SMJs](#).

Dosa

- Use data-aware optimized partitioning.
- Only use **real-time** metrics with grouping entities that are guaranteed to be complete in runtime. (c.f. Runtime partitioning). Example: partitioning key = country, then if users are segregated by country, they can still be used as grouping entities. Partition key = CustomerID, then only customers or other entities that are not shared across customers can be used as grouping entities in real-time metrics.



Railgun supports long time windows

These restrictions do not apply when using the [Railgun engine](#).

- Avoid creating metrics with very long time windows to mimic "write-once streaming" features (e.g. "Time since the first time this customer used this device").
- Don't use very large windows. For very large volumes, the rule of thumb is to use 1 month as the maximum window size.